Quantization

INT8 · INT4 · GPTQ · AWQ · GGUF

How to Run 70B Models on a Single GPU

WEEK 2 · DAY 8 **Monday, July 13 2026** **Day 8 of 84**

#	Topic	What You Learn
01	The Precision Stack	FP32 → FP16 → BF16 → INT8 → INT4
02	PTQ vs QAT	Two philosophies, when to use each
03	GPTQ	Hessian-based layer-wise quantization
04	AWQ	Activation-aware weight quantization
05	GGUF / llama.cpp	CPU & edge inference, naming conventions
06	bitsandbytes	INT8 + NF4 inside HuggingFace
07	Hardware Matrix	Which format for which GPU budget
08	FAANG in Production	Meta ExecuTorch · Apple Core ML · Google TPU
09	Architect's Lens	Decision framework for quantization
10	Hands-On Exercises	Memory audit · benchmark · scenario drill

TEXT-TO-SQL CAPSTONE THREAD — Week 2 introduces the quantization layer. Piece 2 (Schema RAG) arrives in Week 3. Today's content explains HOW the quantized inference engine we will serve SQL through actually works.

Section 1 — The Memory Problem

GPT-3 has 175 billion parameters. Each parameter stored as a 32-bit float occupies 4 bytes. That is 700 GB of VRAM just to load the model — before a single token is processed. Quantization is the discipline of shrinking models so they run on hardware that actually exists.

Numeric Formats: The Precision Stack

Format	Bytes/Param	Range	Primary Use
FP32	4 bytes	±3.4 × 10^38	Training (full precision)
FP16	2 bytes	±65,504	GPU inference (2017+)
BF16	2 bytes	±3.4 × 10^38	TPU/A100 training (Google)
INT8	1 byte	-128 to 127	Production inference
INT4	0.5 bytes	–8 to 7	Edge / memory-constrained

BFloat16 (BF16) was invented at Google Brain for TPUs. It keeps the same 8-exponent-bit range as FP32 (avoiding the NaN overflow that plagues FP16 training) but uses only 7 mantissa bits. BF16 is now the default training dtype on A100/H100.

Memory Impact — Real Model Sizes

Format	Bytes/Param	LLaMA 3 8B	LLaMA 3 70B	LLaMA 3.1 405B
FP32	4	32 GB	280 GB	1,620 GB
FP16	2	16 GB	140 GB	810 GB
INT8	1	8 GB	70 GB	405 GB
INT4	0.5	4 GB	35 GB	~202 GB

> *KEY INSIGHT: LLaMA 3 70B at INT4 fits on ONE A100 80GB GPU. At FP16 it needs two A100s plus inter-GPU communication overhead. That is the practical power of quantization.*

Section 2 — PTQ vs QAT: Two Philosophies

Post-Training Quantization (PTQ)

Take an already-trained FP16 model and quantize it. No retraining.

- Fast and cheap — runs on any downloaded model
- Requires a small calibration dataset (128–512 sentences)
- Some quality loss — the model was not optimized for reduced precision
- Standard approach: GPTQ, AWQ, GGUF, bitsandbytes all do PTQ

Quantization-Aware Training (QAT)

Simulate quantized arithmetic during training, so the model learns to work within the precision constraints.

- Near-lossless quality at INT8/INT4
- Expensive — requires full or partial retraining
- Used by Google for TPU deployment, Apple for Core ML on-device models
- Amazon uses QAT for AWS Inferentia chip optimization

> *For this course and for 99% of production deployments: you will use PTQ. You are working with pre-trained models. QAT is the province of model publishers (Meta, Google, Mistral) who bake it in before release.*

Section 3 — GPTQ: Hessian-Based Layer-Wise Quantization

Paper: *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers* (Frantar et al., 2022).

The Core Insight

Naive INT4 quantization — round every weight to the nearest 4-bit value — destroys quality because some weights have disproportionate influence on the model output. GPTQ asks: given that we must quantize these weights, what quantized values minimize the output error?

The Algorithm

- Take a calibration dataset (128–512 sentences is enough)
- Process one transformer layer at a time
- For each weight matrix W, compute the Hessian — second-order sensitivity of the output to each weight
- Quantize weights in order of Hessian influence, compensating remaining weights after each step to absorb rounding error
- This is Optimal Brain Quantization (OBQ) adapted for speed

Why It Works

If you quantize weight w1 and introduce error e, you nudge the remaining unquantized weights in that row slightly to cancel e out. By the time you quantize the last weight, the accumulated errors have been absorbed. The output of the layer is nearly identical to FP16.

GPTQ Code

```
# pip install auto-gptq
from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig

quantize_config = BaseQuantizeConfig(
    bits=4,           # 4-bit quantization
    group_size=128,   # quantize groups of 128 weights together
    desc_act=False,   # act-order: False = faster; True = slightly better quality
)

model = AutoGPTQForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    quantize_config=quantize_config
)

# Run calibration (128 sentences minimum)
examples = [tokenizer("The quick brown fox", return_tensors="pt")]
model.quantize(examples)
model.save_quantized("Llama-3.1-8B-GPTQ-4bit")
```

Quality at 4-bit — LLaMA 2 7B, WikiText-2 Perplexity

Perplexity measures how surprised the model is by test text — lower is better.

Precision	Perplexity	vs FP16 Baseline
FP16	5.47	baseline
INT4 GPTQ	5.52	+0.9%
INT3 GPTQ	5.89	+7.7%

INT2 GPTQ	10.67	+95%

> *INT4 GPTQ costs less than 1% quality on a 7B model while saving 75% VRAM. INT2 crosses the degradation cliff — avoid it.*

Section 4 — AWQ: Activation-Aware Weight Quantization

Paper: *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration* (Lin et al., 2023, MIT).

The Key Observation GPTQ Misses

1% of weights are responsible for most quantization error — specifically the weights corresponding to activation channels with large magnitudes. When activation xi is large, the product xi * wi dominates the neuron output. Rounding wi by even a tiny amount creates a large absolute error.

AWQ's Solution: Activation Scaling

Before quantizing, find the 'salient' input channels — those with the largest activation magnitudes across the calibration set. Scale UP the weights for those channels by factor s > 1, and scale DOWN the inputs by 1/s.

```
# Mathematically equivalent transformations:
# Original:   output = X * W          (quantize W naively)
# AWQ:        output = (X / s) * (W * s)

# For salient channels: W*s has larger values -> quantization
# grid is finer -> less relative error for important weights.
# The scaling factor s is folded into the preceding LayerNorm,
# so inference speed is unchanged.
```

AWQ vs GPTQ Comparison

Property	GPTQ	AWQ
Method	Hessian-based layer-wise	Activation-scaled PTQ
Quality at 4-bit	Excellent	Slightly better
Inference speed	Fast	Fast (equivalent)
Calibration needed	Yes	Yes
Supported hardware	GPU	GPU + Apple Silicon
Format	.safetensors + quant cfg	.safetensors + AWQ cfg

```
# pip install autoawq
from awq import AutoAWQForCausalLM
```

```
from transformers import AutoTokenizer

model = AutoAWQForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

quant_config = {
    "zero_point": True,
    "q_group_size": 128,
    "w_bit": 4,
    "version": "GEMM"
}
model.quantize(tokenizer, quant_config=quant_config)
model.save_quantized("Llama-3.1-8B-AWQ")
```

> *AWQ is the current recommendation for GPU production deployment. Most HuggingFace LLaMA/Mistral releases include AWQ variants. When you see 'TheBloke' or 'bartowski' repos, they ship AWQ.*

Section 5 — GGUF: Quantization for CPU and Edge

GGUF (GPT-Generated Unified Format) is the file format used by llama.cpp — the C++ library that runs LLMs on CPUs, including MacBooks. Originally GGML (2022), renamed GGUF in 2023: single-file, self-describing, metadata-rich. llama.cpp supports GPU offloading: push some layers to GPU, rest on CPU RAM.

GGUF Quantization Levels

Name	Bits	Size (7B model)	Quality
Q2_K	2.6	~2.8 GB	Noticeable degradation — testing only
Q3_K_M	3.1	~3.5 GB	Acceptable for experimentation
Q4_0	4.0	~3.9 GB	Good, fastest CPU throughput
Q4_K_M	4.5	~4.1 GB	RECOMMENDED: best balance
Q5_K_M	5.7	~4.8 GB	High quality, slightly larger
Q6_K	6.6	~5.5 GB	Near FP16 quality
Q8_0	8.0	~6.7 GB	Essentially lossless
F16	16	~14 GB	Full FP16 — no quantization

Naming Convention Decoded

- Q4 = 4-bit quantization
- _K = k-means quantization (smarter clustering of weight values)
- _M = medium — mixed precision: some layers quantized more aggressively
- _S = small (more aggressive compression)
- _L = large (higher quality, larger file)

Running GGUF Locally

```
# Install llama.cpp
brew install llama.cpp

# Download a GGUF model
huggingface-cli download bartowski/Meta-Llama-3.1-8B-Instruct-GGUF \
    --include "*Q4_K_M*" --local-dir ./models

# Run inference (CLI)
```

```
llama-cli -m models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf \
    -p "Explain quantization in one paragraph" -n 200

# Python via llama-cpp-python
from llama_cpp import Llama

llm = Llama(
    model_path="models/Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf",
    n_ctx=4096,
    n_gpu_layers=32,  # offload 32 layers to GPU; rest stays on CPU RAM
)
output = llm("Explain quantization:", max_tokens=200)
print(output["choices"][0]["text"])
```

Why GGUF matters for Text-to-SQL prototyping: every quantized model on HuggingFace in GGUF format runs locally — no API cost, no network latency, complete privacy. For natural language querying over internal databases, GGUF enables air-gapped deployment.

Section 6 — bitsandbytes: Quantization Inside HuggingFace

bitsandbytes is the most common way to load quantized models directly through HuggingFace transformers. No separate conversion step — one parameter change at model load time.

LLM.int8() — Tim Dettmers, 2022

Key insight: weight matrices in large LLMs have outlier features — a few dimensions that always have large magnitudes. These outliers destroy INT8 quality in naive quantization.

LLM.int8() uses mixed-precision decomposition:

- Identify outlier columns (those exceeding magnitude threshold, default 6.0)
- Keep those columns in FP16
- Quantize all other columns to INT8
- Run two matrix multiplications (FP16 + INT8) and add the results

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    load_in_8bit=True,      # LLM.int8() mixed-precision decomposition
    device_map="auto"       # automatically spread across available GPUs
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")
```

NF4 — NormalFloat4 for QLoRA

NF4 is a 4-bit format designed for weight distributions that follow a normal (Gaussian) distribution — which most LLM weight matrices approximately do. Instead of evenly spaced quantization levels (INT4: -8,-7,...,7), NF4 places more quantization levels near zero (where most weights cluster) and fewer at the extremes. This minimizes average quantization error.

```
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
import torch

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.bfloat16,  # compute in BF16, store as 4-bit
    bnb_4bit_use_double_quant=True,         # quantize scale factors too (saves
                                             # ~0.37 bits/param more)
    bnb_4bit_quant_type="nf4"              # NormalFloat4 — for QLoRA Day 9
)
```

```
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    quantization_config=quantization_config,
    device_map="auto"
)

inputs = tokenizer("What is quantization?", return_tensors="pt").to("cuda")
outputs = model.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

> *DOUBLE QUANTIZATION: setting bnb_4bit_use_double_quant=True quantizes the scale factors themselves (FP32 -> FP8), saving an additional ~0.37 bits per parameter. On a 7B model that's ~300MB extra savings.*

NF4 is the foundation for QLoRA (Quantized LoRA) — Day 9's topic. You load the base model in NF4 and attach trainable LoRA adapters on top. The 4-bit base stays frozen; only the small adapters (0.1% of params) train.

Section 7 — Hardware Decision Matrix

Format Selection by Use Case

Use Case	Best Format	Why
GPU serving, production API	AWQ 4-bit	Best quality+speed on GPU
GPU fine-tuning (QLoRA)	NF4 via bitsandbytes	Works with LoRA adapters
CPU inference / local dev	GGUF Q4_K_M	Runs anywhere, no CUDA
Memory debug / near-lossless	INT8 bitsandbytes	One line, ~99.9% quality
Mobile / on-device	INT4 Core ML / TFLite	Platform-optimized kernels

VRAM Requirements — LLaMA 3.1 Family

Model	FP16 VRAM	INT8 VRAM	AWQ 4-bit VRAM	Hardware (INT4)
8B	16 GB	8 GB	5 GB	RTX 4070 / M2 Pro
70B	140 GB	70 GB	38 GB	Single A100 80GB (with KV cache room)
405B	810 GB	405 GB	~210 GB	3x A100 80GB minimum

The Counter-Intuitive Quality Rule

Model	FP16 Perplexity	INT4 Perplexity	vs 7B FP16
LLaMA 2 7B	5.47	5.52	baseline
LLaMA 2 13B	4.88	4.91	+12% better
LLaMA 2 70B	3.32	3.35	+52% better

ARCHITECT RULE: A 70B model at INT4 (PPL 3.35) massively outperforms a 7B model at FP16 (PPL 5.47). Larger models are more robust to quantization. Always quantize the biggest model that fits your VRAM budget — do NOT compromise on model size to preserve precision.

Section 8 — FAANG in Production

Meta — LLaMA On-Device (iOS/Android)

Meta ships LLaMA models in 4-bit quantization for on-device inference via the ExecuTorch framework. A 1B-3B model quantized to 4-bit fits in 2 GB of phone RAM. The quantization pipeline uses PTQ with per-channel INT4 plus mixed INT8 for sensitive layers. Meta's 'AI features' in WhatsApp and Instagram run entirely on-device in low-precision format.

Apple — Core ML INT4

Apple's Core ML Tools converts models to INT4 for Apple Silicon deployment (iPhone, MacBook with Neural Engine). The MLX framework — Apple's ML framework for M-series chips — natively supports 4-bit quantization with hardware-accelerated kernels. When you run Apple Intelligence features, the on-device model is INT4 or INT8 via Core ML.

Google — TPU INT8 + BF16 Training

Google TPUs have INT8 multiply-accumulate units built into the hardware. Training on TPUs happens in BF16, but inference is deployed in INT8 to maximize throughput per TPU dollar. The Gemma models (Google open models) are quantized and released in GGUF format for community use. Gemini API inference on Google's infrastructure uses chip-level INT8.

Amazon — AWS Inferentia Chip

Amazon's Inferentia chips (used in Bedrock for cost-efficient serving) have hardware INT8 and INT4 support. Models deployed on Inferentia go through the AWS Neuron SDK quantization pipeline before deployment. This is how Bedrock achieves the price points advertised for Amazon Titan and third-party models — INT8 inference on specialized hardware.

Netflix — Real-Time Recommendation Inference

Netflix's recommendation models run inference on billions of requests per day. Serving full-precision models at that scale is cost-prohibitive. Netflix uses INT8 quantization for embedding lookup and shallow transformer layers in their recommendation stack, with FP16 for the deeper attention layers that affect quality most. The result: same recommendation quality at a fraction of the inference cost.

Section 9 — Architect's Lens

The quantization decision is a joint optimization over model size, hardware budget, latency requirements, and quality constraints. Follow this framework:

```
DECISION FRAMEWORK: Which quantization should I use?

1. Do I have GPU VRAM?
   NO  -> GGUF Q4_K_M via llama.cpp (CPU, runs anywhere)
   YES -> Continue

2. Am I fine-tuning the model?
   YES -> bitsandbytes NF4 (QLoRA-compatible, see Day 9)
   NO  -> Continue

3. What is my priority?
   MAX QUALITY  -> AWQ 4-bit (best compression/quality ratio)
   MAX SPEED    -> GPTQ 4-bit with desc_act=False
   SIMPLICITY   -> INT8 bitsandbytes (one flag, near-lossless)

4. Is the model already quantized on HuggingFace?
   YES -> Download it directly (bartowski/*, TheBloke/*)
   NO  -> Run quantization yourself (~2hrs for 70B on one A100)

5. How large is the model?
   <10B  -> INT8 is lossless enough; simpler than INT4
   10B+  -> INT4 to fit available hardware
   >100B -> INT4 mandatory; consider multi-GPU + pipeline parallelism
```

The Trap Architects Fall Into

Optimizing precision before choosing model size. A 70B INT4 model beats a 7B FP16 model in quality AND often fits in the same or less total VRAM. Size and precision are jointly optimized, not independent choices.

Latency Profile by Precision

Precision	Memory Bound?	Compute Bound?	Best For
FP16	Yes (large model)	Yes	Batch serving, high throughput
INT8	No (fits better)	Slightly more	Memory-constrained single GPU
INT4	No (much smaller)	Compute bound	Throughput + cost optimization
GGUF	RAM bandwidth	CPU limited	Local/edge, no GPU

On small models (<7B), INT8/INT4 can actually be SLOWER than FP16 — the overhead of dequantization dominates when compute is not the bottleneck. On 70B+ models, INT4 is always faster because memory bandwidth is the limit and smaller weights transfer faster.

Section 10 — Text-to-SQL Thread: The Inference Engine

Piece 1 (Day 6) established the SQLOutput structured output schema — the model produces validated, typed SQL. Today's lesson explains the engine that will run our SQL-generation model at production scale.

Architecture Preview

```
User query
     |
     v
[Schema RAG retriever]  <-- Week 3, Piece 2
     |
     v
[Quantized SQL-gen model]  <-- TODAY: this layer
     |                          (70B AWQ 4-bit on one A100)
     v
[SQLOutput validator]       <-- Piece 1 (Day 6)
     |
     v
Database (StarRocks / BigQuery / Snowflake)
```

Why Quantization Matters Here Specifically

- A 70B model fine-tuned for SQL generation at AWQ 4-bit uses 38 GB VRAM — fits on one A100 80GB with 42 GB headroom for KV cache
- FP16 would need two A100s plus NVLink interconnect — 2x the infrastructure cost
- Latency: AWQ 4-bit generates SQL in ~200ms vs ~500ms for FP16 (memory bandwidth)
- At 10,000 analyst queries/day: ~2ms difference per query = meaningful SLA gap

Quantized Inference for Text-to-SQL

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from pydantic import BaseModel
from typing import Literal
import json, torch

# Load AWQ-quantized SQL-generation model
model = AutoModelForCausalLM.from_pretrained(
    "your-org/sql-llama-70b-awq",       # AWQ pre-quantized
    device_map="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained("your-org/sql-llama-70b-awq")
```

```
# From Day 6: structured output schema
class SQLOutput(BaseModel):
    sql: str
    tables_used: list[str]
    confidence: Literal["high", "medium", "low"]

def generate_sql(natural_query: str, schema_context: str) -> SQLOutput:
    prompt = f"""Schema:
{schema_context}

Question: {natural_query}

Respond with JSON only, format:
{{"sql": "...", "tables_used": [...], "confidence": "high|medium|low"}}"""

    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs, max_new_tokens=256,
            temperature=0.1,  # low temp for deterministic SQL
            do_sample=False,
        )
    raw = tokenizer.decode(outputs[0][inputs.input_ids.shape[1]:],
                           skip_special_tokens=True)
    return SQLOutput(**json.loads(raw))
```

NEXT: Week 3 builds Piece 2 — Schema RAG. Instead of passing the full schema (which can be 100K+ tokens for large data warehouses), we retrieve only the relevant tables and columns using embeddings. The quantized model then generates SQL against a focused, accurate context.

Section 11 — Hands-On Exercises

Exercise 1: Memory Audit (5 minutes)

Run this to understand your hardware budget before choosing a model:

```
import torch

if torch.cuda.is_available():
    for i in range(torch.cuda.device_count()):
        props = torch.cuda.get_device_properties(i)
        vram_gb = props.total_memory / 1e9
        print(f"GPU {i}: {props.name}, {vram_gb:.1f} GB VRAM")
        print(f"  Max model size at FP16: {vram_gb / 2:.0f}B params")
        print(f"  Max model size at INT4: {vram_gb / 0.5:.0f}B params")
elif torch.backends.mps.is_available():
    import psutil
    ram_gb = psutil.virtual_memory().total / 1e9
    print(f"Apple Silicon: {ram_gb:.0f} GB unified RAM")
    avail = (ram_gb - 8) / 0.5  # leave 8GB for OS
    print(f"  Recommended max: {avail:.0f}B params at INT4")
else:
    import psutil
    ram_gb = psutil.virtual_memory().total / 1e9
    print(f"CPU only. Use GGUF Q4_K_M.")
    print(f"  System RAM: {ram_gb:.0f} GB -> max ~{ram_gb * 0.6:.0f}B params at INT4")
```

Exercise 2: Benchmark FP16 vs INT8 vs NF4

Run a small model at three precision levels and compare VRAM + quality:

```
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch, time

MODEL = "meta-llama/Llama-3.2-1B-Instruct"  # small model
PROMPT = "Explain quantization in machine learning in 3 sentences."

tokenizer = AutoTokenizer.from_pretrained(MODEL)
inputs = tokenizer(PROMPT, return_tensors="pt")

def run_model(model_kwargs, label):
    model = AutoModelForCausalLM.from_pretrained(
        MODEL, device_map="auto", **model_kwargs)
    start = time.time()
    with torch.no_grad():
        out = model.generate(
            inputs.input_ids.to(model.device), max_new_tokens=80)
    latency = time.time() - start
    text = tokenizer.decode(out[0][inputs.input_ids.shape[1]:],
```

```
                          skip_special_tokens=True)
    vram = (torch.cuda.max_memory_allocated() / 1e9
            if torch.cuda.is_available() else 0)
    print(f"\n=== {label} ===")
    print(f"VRAM: {vram:.2f} GB | Latency: {latency:.2f}s")
    print(f"Output: {text[:200]}")
    if torch.cuda.is_available():
        torch.cuda.reset_peak_memory_stats()
    del model

# Run all three
run_model({"torch_dtype": torch.float16}, "FP16 Baseline")
run_model({"load_in_8bit": True}, "INT8 (LLM.int8())")
bnb = BitsAndBytesConfig(load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True, bnb_4bit_quant_type="nf4")
run_model({"quantization_config": bnb}, "NF4 4-bit")
```

Exercise 3: Scenario Drill (No Code Required)

Answer each based on this lesson — architect-level reasoning:

Scenario A: Run LLaMA 3 70B on one NVIDIA RTX 4090 (24 GB VRAM).

Answer: AWQ 4-bit (~38 GB) still exceeds 24 GB. Use GGUF Q4_K_M with n_gpu_layers to offload as many layers as fit, keeping rest in system RAM.

Scenario B: Fine-tune Mistral 7B on an M2 MacBook Pro (32 GB RAM).

Answer: bitsandbytes NF4 (QLoRA) — loads model in 4-bit, trains only LoRA adapters. On Apple Silicon use MLX or bitsandbytes with MPS device.

Scenario C: Serve 10K Text-to-SQL requests/day with 70B model on one A100 80GB.

Answer: AWQ 4-bit. 38 GB model + 42 GB headroom for large KV cache. Fastest GPU inference and best quality among 4-bit options.

Scenario D: User wants a local chatbot with no GPU at all.

Answer: GGUF Q4_K_M via llama.cpp or Ollama. Runs on CPU + system RAM. 8B model at Q4_K_M = ~4 GB RAM — runs on any modern laptop.

Summary — Day 8 Key Concepts

Concept	One-Line Definition
FP32 -> INT4	Fewer bits per weight = less VRAM, some quality loss
PTQ	Quantize after training — fast, good enough for most cases
QAT	Simulate quantization during training — better quality, expensive
GPTQ	Hessian-based layer-wise PTQ, standard for GPU 4-bit
AWQ	Activation-scaled PTQ, slightly better quality than GPTQ
GGUF	CPU-friendly format from llama.cpp, runs anywhere
bitsandbytes	HuggingFace-integrated INT8 + NF4 quantization
NF4	4-bit format for normally-distributed weights; used in QLoRA
Perplexity	Quality metric: lower = better; INT4 costs <1% on large models
70B > 7B at INT4	Bigger model + lower precision still beats smaller model at FP16

TOMORROW — Day 9: LoRA + PEFT

LoRA (Low-Rank Adaptation) lets you fine-tune a 70B model by training only 0.1% of its weights. Combined with today's NF4 quantization, QLoRA makes full-power fine-tuning possible on a single consumer GPU. You will build a LoRA adapter for SQL generation on top of a quantized base model.

EAGLE EYE ADDENDUM — DAY 8 | Quantization Theory: absmax · zero-point · symmetric · asymmetric · W4A16 · W8A8

Day 8 covered INT8, INT4, GPTQ, AWQ, QLoRA, NF4, GGUF, and bitsandbytes in detail. Six foundational quantization theory terms were missing: absmax, zero-point, symmetric, asymmetric, W4A16, and W8A8. These are the building blocks that explain HOW every quantization format works internally.

The Two Families of Quantization: Symmetric vs Asymmetric

Every quantization method reduces a float (FP32/BF16) to a lower-precision integer using a linear mapping. The two fundamental approaches differ in whether the quantization range is centred at zero.

```
# ■■ SYMMETRIC QUANTIZATION (absmax) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Maps [-max_abs, +max_abs] → [-127, 127]  (INT8 symmetric)
# Scale factor: s = max(|w|) / 127   (absmax = absolute maximum)
# Quantize:  q = round(w / s)
# Dequantize: w_approx = q * s
# Zero point: always 0  (no offset needed — symmetric around zero)

import torch

def absmax_quantize_int8(weights: torch.Tensor):
    scale = weights.abs().max() / 127.0
    q = (weights / scale).round().clamp(-127, 127).to(torch.int8)
    return q, scale

def absmax_dequantize_int8(q: torch.Tensor, scale: float):
    return q.float() * scale

# Example:
w = torch.tensor([0.1, -0.5, 0.8, -0.2, 0.3])
q, s = absmax_quantize_int8(w)
print(f"Scale: {s:.4f}")
print(f"Quantized:   {q.tolist()}")   # [-16, 79, -127, 32, -47] approx
print(f"Dequantized: {absmax_dequantize_int8(q, s).tolist()}")
# Error is small — only the rounding step introduces error

# USED BY: LLM.int8() (bitsandbytes INT8), GPTQ, most INT8 schemes
# BEST FOR: weights (roughly symmetric distribution around zero)
```

```
# ■■ ASYMMETRIC QUANTIZATION (zero-point) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
# Maps [min_val, max_val] → [0, 255]  (UINT8 asymmetric, or [-128,127] INT8)
# Scale:      s = (max_val - min_val) / 255.0
# Zero point: z = round(-min_val / s)   ← the KEY difference from symmetric
# Quantize:   q = round(w / s) + z
# Dequantize: w_approx = (q - z) * s

def zeropoint_quantize_uint8(weights: torch.Tensor):
    min_val = weights.min().item()
```

```
    max_val = weights.max().item()
    scale     = (max_val - min_val) / 255.0
    zero_point = round(-min_val / scale)
    q = (weights / scale + zero_point).round().clamp(0, 255).to(torch.uint8)
    return q, scale, zero_point

def zeropoint_dequantize(q, scale, zero_point):
    return (q.float() - zero_point) * scale

# Example:
w = torch.tensor([0.0, 0.2, 0.5, 0.8, 1.0])   # ReLU output — all positive
q, s, zp = zeropoint_quantize_uint8(w)
print(f"Scale: {s:.4f}, Zero point: {zp}")
print(f"Quantized: {q.tolist()}")
print(f"Dequantized: {zeropoint_dequantize(q, s, zp).tolist()}")

# USED BY: activation quantization (activations are often non-symmetric, e.g. ReLU output)
# BEST FOR: activations, embeddings, any tensor with non-zero minimum
```

Property	Symmetric (absmax)	Asymmetric (zero-point)
Range	[-max_abs, +max_abs]	[min_val, max_val]
Zero point	Always 0 (no offset)	Non-zero offset z
Formula	q = round(w / s)	q = round(w/s) + z
INT range	[-127, 127] INT8	[0, 255] UINT8 or [-128, 127]
Best for	Weights (symmetric)	Activations (often skewed)
Used by	GPTQ, bitsandbytes INT8	PyTorch QAT, TensorRT INT8
Overhead	None (z=0, no subtract)	Extra subtract of z at inference

W4A16 and W8A8: The Standard Notation

In quantization literature and production systems, formats are described using WxAy notation: W = weight precision, A = activation precision. This notation tells you exactly what is quantized and what stays in full precision.

```
NOTATION GUIDE:
  WxAy = x-bit weights, y-bit activations
  W = weights (model parameters — large, static, quantized offline)
  A = activations (computed at runtime — dynamic, harder to quantize)

W4A16 — 4-bit weights, 16-bit activations
  Weights stored as INT4 (0.5 bytes/param)
  Activations computed in FP16 at runtime — no activation quantization error
  Dequantize weights to FP16 just before each matmul; then compute in FP16
  Memory: 4× smaller than FP16 (just weights)
  Compute: FP16 matmuls (GPU-native, fast)
  Quality: 98-99% of FP16 quality on most tasks
  Used by: AWQ, GPTQ, QLoRA (training), NF4 (bitsandbytes)
```

```
W8A8 — 8-bit weights AND 8-bit activations
  Weights stored as INT8; activations also quantized to INT8 before matmul
  Matmuls execute as INT8 x INT8 → INT32 accumulate → dequantize
  Hardware benefit: INT8 matmuls on A100/H100 are 2x faster than FP16
  Memory: 2× smaller than FP16
  Compute: INT8 matmuls (fastest path on modern GPUs)
  Quality: 96-99% of FP16 quality (harder than W4A16 — activations vary)
  Used by: bitsandbytes LLM.int8(), TensorRT-LLM INT8, NVIDIA Transformer Engine
  Challenge: activations have outlier features that must be handled (LLM.int8() uses
             mixed-precision: outlier channels stay in FP16, rest go INT8)

W16A16 — baseline (no quantization, both in FP16)
W4A4  — aggressive 4-bit both (research only; significant quality loss)
W2A16 — 2-bit weights (very aggressive; used in some edge scenarios)
```

```
# Practical examples of each format in use:

# W4A16 — AWQ (most common production choice):
from awq import AutoAWQForCausalLM
model = AutoAWQForCausalLM.from_quantized(
    "TheBloke/Llama-3-8B-Instruct-AWQ",
    fuse_layers=True,               # fuses dequant + matmul for speed
)
# Weights: INT4; activations: FP16 at runtime; matmul: FP16

# W4A16 — GPTQ (alternative, slightly lower quality than AWQ):
from auto_gptq import AutoGPTQForCausalLM
model = AutoGPTQForCausalLM.from_quantized("TheBloke/Llama-3-8B-GPTQ-4bit")

# W8A8 — bitsandbytes LLM.int8():
import transformers
model = transformers.AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.1-8B-Instruct",
    load_in_8bit=True,              # W8A8 with mixed-precision for outliers
    device_map="auto",
)
# Outlier channels (>6 std devs) kept in FP16; rest INT8
# Result: ~0.1% quality loss at 2x memory savings vs FP16
```

Notation	W bits	A bits	Memory vs FP16	Speed	Quality	Format Example
W16A16	16	16	1× (baseline)	FP16 matmul	100%	BF16 baseline
W8A8	8	8	2× smaller	INT8 matmul (fastest)	98-99%	LLM.int8()
W4A16	4	16	4× smaller	FP16 matmul	98-99%	AWQ, GPTQ, NF4
W4A4	4	4	4× smaller	INT4 matmul	94-96%	Research only
W2A16	2	16	8× smaller	FP16 matmul	90-94%	Edge/extreme

WHY W4A16 DOMINATES PRODUCTION: Activations are hard to quantize because they vary per input and have outlier values that cause large quantization errors. Weights are static and pre-quantized offline where you can run calibration to minimise error. W4A16 gets the 4× memory win (the expensive part — weights are 95% of model size) while avoiding activation quantization error entirely. This is why AWQ and GPTQ (both W4A16) are the dominant production formats at Meta, Google, Amazon, and Netflix for serving quantized models.

Eagle Eye Addendum — Day 8: Weight Compression

Weight compression is the overarching term for reducing the storage size of model weights. Quantization is the most common technique — reducing float32 (4 bytes/param) to int8 (1 byte) or int4 (0.5 bytes) achieves 4x–8x weight compression with minimal quality loss.

```
# Weight compression achieved through quantization:

# LLaMA-3 70B example:
# FP32:  70B params × 4 bytes = 280 GB  (baseline, no compression)
# BF16:  70B params × 2 bytes = 140 GB  (2x weight compression)
# INT8:  70B params × 1 byte  =  70 GB  (4x weight compression, via bitsandbytes)
# INT4:  70B params × 0.5 byte=  35 GB  (8x weight compression, via GPTQ/AWQ)
# GGUF Q4_K_M: ~39 GB         (mixed-precision weight compression for CPU)

# Weight compression ratio formula:
original_bytes  = n_params * bytes_per_param_fp32  # e.g., 70e9 * 4 = 280e9
compressed_bytes= n_params * bytes_per_param_quant # e.g., 70e9 * 1 =  70e9
compression_ratio = original_bytes / compressed_bytes  # e.g., 4.0x

# Key insight: weight compression reduces GPU memory requirement,
# enabling larger models to fit on fewer GPUs (or consumer hardware).
```